

Class 8 — Angular Template-Driven Forms

Topic

A fully functional **Edit Book** form using Angular's Template-Driven Form approach.

- ☐ ☒ Loads existing book data from the API when an `id` route param is present
- ☐ ☒ Binds all form fields to a `Book` object via `[(ngModel)]`
- ☐ ☒ Groups nested publisher fields with `ngModelGroup`
- ☐ ☒ Saves via **PUT** (update) or **POST** (create) based on whether `book.id` exists
- ☐ ☒ Navigates back to `/books` after a successful save
- ☐ ☒ Prevents double-submission by disabling the Save button on submit

Core Concept 1 — FormsModule Setup

 **To use Template-Driven Forms**, you must import `FormsModule` into the component. This unlocks `ngForm`, `ngModel`, and `ngModelGroup` directives in the template.

```
// edit-book.ts
import { FormsModule, NgForm } from '@angular/forms';

@Component({
  imports: [RouterLink, FormsModule], // ← FormsModule required
})
export class EditBook {}
```

Core Concept 2 — Template Reference Variables

A **template reference variable** (`#varName`) gives you a local handle on a DOM element or a directive inside the template.

```
<!-- #h1 → plain DOM element reference -->
<h1 #h1>Edit Book</h1>
<button (click)="showH1(h1)">click me</button>

<!-- #form="ngForm" → NgForm directive instance (not the element) -->
<form (ngSubmit)="saveBook(form)" #form="ngForm">
```

Syntax	Points to
<code>#h1</code>	Native <code>HTMLHeadingElement</code> DOM node
<code>#form="ngForm"</code>	<code>NgForm</code> directive instance (form state, value, validity)

```
showH1(h1: HTMLHeadElement) {
  console.log(h1);      // the real DOM element
}

saveBook(form: NgForm) {
  console.log(form);    // form.value / form.valid / form.submitted
}
```

 **Key distinction:** `#form` alone gives you the `<form>` HTML element. `#form="ngForm"` exports the **directive** — that is what gives you `.value`, `.submitted`, `.controls`, etc.

Core Concept 3 — `[(ngModel)]` Two-Way Binding

`[(ngModel)]` is the core of Template-Driven Forms. It **syncs** the input and the component property in both directions.

```
<input placeholder="Book Name" [(ngModel)]="book.name" name="name"/>
<span>{{ book.name }}</span> <!-- updates live as you type -->
```

How it works:

- User types → input fires `(ngModelChange)` → `book.name` updates in the component
- Code changes `book.name` → Angular re-renders the input value

⚠ **Golden rule:** Every `[(ngModel)]` element inside a `<form>` **must** have a `name` attribute. Angular uses `name` as the key to register the control on `NgForm`.

```
<!-- ✅ Correct -->
<input [(ngModel)]="book.author" name="author"/>

<!-- ❌ Runtime error – name attribute missing -->
<input [(ngModel)]="book.author"/>
```

The `[(banana-in-box)]` syntax is shorthand for:

```
<input [(ngModel)]="book.name" (ngModelChange)="book.name = $event" name="name"/>
```

🧩 Core Concept 4 — `ngModelGroup` (Nested Form Groups)

`ngModelGroup` wraps a set of controls into a **named sub-group** inside the form model — mirroring a nested object in your TypeScript model.

```
<section ngModelGroup="publisher">
  <input [(ngModel)]="book.publisher.publisherName" name="publisherName"/>

  <!-- Radio buttons: same name + same ngModel, different value -->
```

```

<label><input type="radio" value="traditional" [(ngModel)]="book.publisher.publisherType" name="publisherType"/> Traditional</label>
<label><input type="radio" value="self-published" [(ngModel)]="book.publisher.publisherType" name="publisherType"/> Self-Published</label>
<label><input type="radio" value="academic" [(ngModel)]="book.publisher.publisherType" name="publisherType"/> Academic</label>
</section>

```

`form.value` will then look like:

```

{
  "name": "Clean Code",
  "author": "Robert Martin",
  "publisher": {
    "publisherName": "Prentice Hall",
    "publisherType": "traditional"
  },
  "category": "Programming"
}

```

💡 **Radio buttons** share the same `name` and `[(ngModel)]` binding. Angular determines which is selected by matching the `value` attribute to the bound property.

🧩 Core Concept 5 — Disable Save on Submit

```

<button type="submit" [disabled]="form.submitted">Save</button>

```

- `NgForm.submitted` flips to `true` the instant the form submits
- `[disabled]="form.submitted"` immediately greys out the button

- Combined with `delay(2000)` (simulated network), this shows students **why** double-submit prevention matters

💡 In production, you'd typically reset `form.submitted` or reset the form after the HTTP response to allow re-submission.

⚙️ BooksService — Smart Save Pattern

```
save(book: Book): Observable<Book> {
  if (book.id) {
    return this.http.put<Book>(`/api/books/${book.id}`, book); // UPDATE
  }
  return this.http.post<Book>(`/api/books`, book); // CREATE
}
```

💡 One `save()` method handles both **create** and **update**. The component doesn't need to know which HTTP verb to use — it just calls `save()`.

Method	HTTP Verb	Endpoint	When
<code>getAllBooks()</code>	GET	<code>/api/books</code>	Fetch all books
<code>getBook(id)</code>	GET	<code>/api/books/:id</code>	Load form data
<code>save(book)</code>	POST	<code>/api/books</code>	<code>book.id</code> is falsy (new)
<code>save(book)</code>	PUT	<code>/api/books/:id</code>	<code>book.id</code> is truthy (existing)

🔬 Component Deep-Dive — `EditBook`

Pre-initializing the Book Object

```
book: Book = {
  id: 0,
  name: '',
  author: '',
  publishedYear: null,
  rating: null,
```

```

publisher: { publisherName: '', publisherType: '' },
category: '',
description: ''
};

```

Loading Existing Data via Route Params

```

ngOnInit() {
  this.route.paramMap.subscribe(paramMap => {
    const bookId = Number(paramMap.get('id'));
    if (!bookId) return; // guard: no id = "create new" flow
    this.getBook(bookId);
  });
}

getBook(id: number) {
  this.bookService.getBook(id).subscribe((res) => {
    this.book = res;
    this.cdr.detectChanges(); // force re-render with new data
  });
}

```



All Form Controls Used

Input Type	HTML Element	Bound To
Text	<code><input></code>	<code>book.name</code> , <code>book.author</code>
Date	<code><input type="date"></code>	<code>book.publishedYear</code>
Number	<code><input type="number"></code>	<code>book.rating</code>
Radio	<code><input type="radio"></code>	<code>book.publisher.publisherType</code>
Select/Dropdown	<code><select></code>	<code>book.category</code>

Input Type	HTML Element	Bound To
Textarea	<code><textarea></code>	<code>book.description</code>

Quick Reference Cheat Sheet

Concept	Syntax	Purpose
Enable TDF	<code>imports: [FormsModule]</code>	Activates all TDF directives
Export form instance	<code>#form="ngForm"</code>	Access NgForm state in template
Two-way binding	<code>[(ngModel)]="prop"</code>	Sync input ↔ component property
Register control	<code>name="fieldName"</code>	Required for NgForm to track the control
Nested group	<code>ngModelGroup="key"</code>	Groups controls into a sub-object
Handle submit	<code>(ngSubmit)="fn()"</code>	Clean form submission
DOM reference	<code>#ref</code>	Access native element
Prevent double-submit	<code>[disabled]="form.submitted"</code>	Disable button after first click